

Testing Improvements

Two independent problems: the pack cannot verify itself, and the testing capability it ships to target projects has structural holes.

Prepared 4 August 2026 Scope installers, `test-warden`, routing map Items 13 proposals

Finding that frames everything below. Of the 13 wardens, only `ticket-warden` is granted `Bash`. Every other agent — including `test-warden` — holds `Read`, `Grep`, `Glob`, `Edit`, `Write`.

So the agent responsible for test quality cannot execute a test. Its own rules 10 and 13 require proving a test fails before it passes; neither is observable without running something. That is item B1, and it is the cheapest high-impact change in this document.

A — The pack cannot verify itself

The product is two installers (~15KB PowerShell, ~6KB bash) that must behave identically. The entire strategy is a human reading `TEST.md` and typing commands. 8 of 20 scenarios remain untested (■).

A1 Make the `TS-<n>` catalog executable

HIGH

`TEST.md` is already written as preconditions → exact commands → concrete expected result. That is a test spec asking to become a harness. Add `tests/` with a small assert helper, one temp directory per case, and teardown.

Keep `TEST.md` as the human-readable catalog, with each scenario referencing its automated counterpart so the two cannot drift apart.

A2 Express parity as a golden-tree diff, not prose

HIGH

Install with each script into two fresh directories, normalize line endings, then `diff -r` the trees.

Why this one first: TS-17 is the scenario most likely to rot silently, because two implementations of the same logic in two languages always drift. A tree diff buys the entire surface in one test — you never have to enumerate what to compare.

A3 Unit-test the commit hook as a pure function

HIGH

`.github/commit-msg` is stdin-file → exit code. No git required, milliseconds to run, table-driven. TS-10 covers the happy path and four exempt prefixes; the interesting cases are unguarded:

- lowercase `sr-1`; `SR-` with no digits
- leading whitespace; empty message
- subject on line 2 after a blank line
- a *body* line that looks like `SR-1`
- UTF-8 BOM

Highest bug-density per byte in the repo, and the cheapest component to cover exhaustively.

A4 Derive baseline counts instead of hardcoding them

MEDIUM

"4 docs, 24 entries, 1 marker pair" is prose in `TEST.md`. Adding a 14th warden makes every count silently wrong. Assert `count(.claude/agents) == count(payload/agents)` instead.

Apply the same to the routing table: assert it names exactly the installed wardens, with an explicit allowlist for deliberate omissions (`ticket-warden` is intentionally unrouted).

A5 Prioritize the 8 open scenarios by blast radius

HIGH

- **TS-8 first** — "`-Force` overwrites agents but never docs" is a *data-loss* invariant. A regression destroys someone's hand-written `SPEC.md`.
- **TS-2 / TS-20 next** — these fail *silently*. The hook is copied but not enabled, and the user believes they are protected when they are not.
- **TS-19** — a corrupt payload leaves the target half-installed.
- **TS-4 / 5 / 7** — cosmetic by comparison.

Sequencing by number treats a data-loss bug and a console-message nit as equals.

A6 Add the negative cases nobody wrote down

MEDIUM

Target path containing spaces or non-ASCII characters (a classic PowerShell quoting failure), read-only target, target identical to the pack directory, deeply nested path.

Precisely where cross-platform installers break, and none are currently represented.

A7 Add CI across both platforms

MEDIUM

A GitHub Actions matrix on `ubuntu-latest` and `windows-latest` running A1 makes "untested by hand" structurally impossible rather than a discipline problem.

Note the repo is not currently a git repository at all — worth resolving independently of CI.

B — The shipped testing capability has holes

What every installed project inherits from `test-warden` and the routing map.

B1 `test-warden` cannot run the tests it writes

CRITICAL

It has no `Bash`. Yet rule 10 requires that "*a new test must fail first — prove the test can catch the bug it guards against*", and rule 13 requires reproducing a bug in a failing test before fixing it. **Neither is verifiable without executing anything.**

The agent writes tests blind and asserts a compliance it has no way to observe.

Fix: grant `test-warden` `Bash` so it can run the suite, watch a new test go red, apply the fix, and watch it go green. Everything else in section B is secondary to this.

B2 Nobody owns integration, e2e, or contract tests

CRITICAL

`test-warden` 's negative scope explicitly disclaims them, and no other warden picks them up. A project installs 13 wardens and inherits an entirely unowned test tier.

Recommendation: a separate `integration-warden` rather than broadening `test-warden` — because the rules genuinely conflict. Rule 4 forbids real network, filesystem, and database; rule 8 forbids framework boot. Those are exactly what an integration test does.

Housing both rule sets in one file invites the agent to apply unit rules to integration tests and the reverse. Largest structural hole, and the most design work.

B3 The curl document is owned by the wrong warden

HIGH

It lives in `test-warden.md:131` but triggers only “whenever your work touches an HTTP API.” When **controller-warden** adds an endpoint and no test changes, the document silently goes stale — despite the section insisting that completeness is the entire point.

Options: move the obligation to controller-warden, who authors the routes; or make it a verified artifact that enumerates routes and diffs against the file.

Hand-maintained completeness does not survive contact with a real project.

B4 Origin-project details leak into a stack-neutral pack

MEDIUM

`http://localhost:5000/api`, `backend-java/.../web`, and the Flask `@app.route` examples are hardcoded in a pack that advertises framework-agnostic rules.

Make them installer-substituted placeholders, or values the agent reads from the conventions section of `CLAUDE.md`.

B5 No React or TypeScript testing examples

MEDIUM

`test-warden-refs/` ships Python and Java. `frontend-warden` ships React. A React project therefore receives frontend *code* rules with no frontend *testing* examples — no React Testing Library, no guidance on querying by role over test-id.

The most straightforward gap in this document to close.

B6 No cross-cutting “changes ship with tests” rule

MEDIUM

security-warden, api-client-warden, and exception-warden all author highly testable contracts — authorization decisions, retry and timeout and circuit-breaker behavior, status-code mapping — yet nothing obligates tests when they write code.

The routing map splits work by layer; the testing obligation needs to cut *across* it, in `CLAUDE.md`.

Recommended sequence

1. **B1** — grant `test-warden` Bash. Smallest change, largest effect on shipped quality.
2. **A2 + A3** — parity tree-diff and hook unit tests. Most regression protection per hour spent on the pack itself.
3. **A1 + A5** — build out the harness, closing the open scenarios in blast-radius order.
4. **B2** — design `integration-warden` . Biggest hole, but genuine design work rather than mechanical change.
5. Remainder — B3 through B6, then A4, A6, A7 as CI comes online.

Line references reflect the working tree at v1.0.0. Item B1 is in progress; the remainder are proposals awaiting decision.